Solution: Number of Closed Islands

Problem Statement

You are given a 2D matrix containing only 1s (land) and 0s (water).

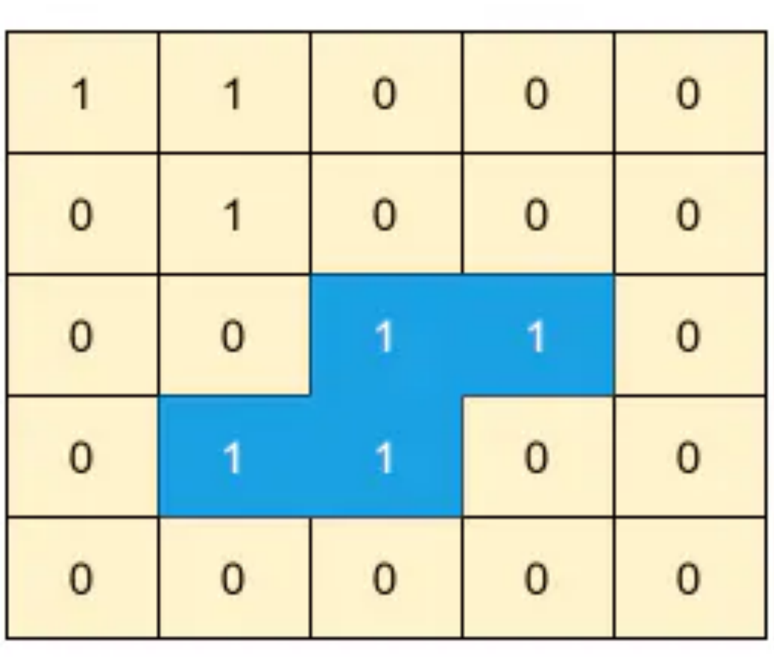
An **island** is a connected set of 1s (land) and is surrounded by either an edge or 0s (water). Each cell is considered connected to other cells horizontally or vertically (not diagonally).

A **closed island** is an island that is totally surrounded by 0s (i.e., water). This means all horizontally and vertically connected cells of a closed island are water. This also means that, by definition, a closed island can't touch an edge (as then the edge cells are not connected to any water cell).

Write a function to find the number of closed islands in the given matrix.

Example 1

Input: matrix =

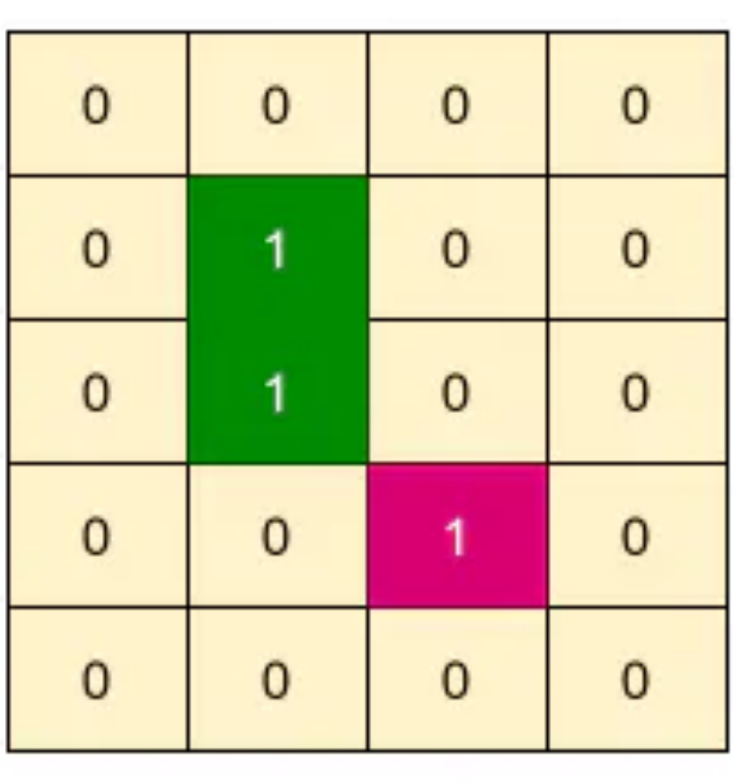


1	1	0	0	0
0	1	0	0	0
0	0	1	1	0
0	1	1	0	0
0	0	0	0	0

Output: 1 Explanation: The given matrix has two islands, but only the highlighted island is a closed island. The other island is touching the boundary that's why is is not considered a closed island.

Example 2

Input: matrix =



0	0	0	0
0	1	0	0
0	1	0	0
0	0	1	0
0	0	0	0

Output: 2

Explanation: The given matrix has two islands and both of them are closed islands.

Solution

The question follows the Island pattern and is quite similar to Number of Islands problem.

We will traverse the matrix linearly to find islands. We can use the Depth First Search (DFS) or Breadth First Search (BFS) to traverse an island i.e., to find all of its connected land cells.

How do we decide if an island is a closed island?

To find that out, while traversing an island we need to ensure two things:

1. The island does not touch an edge.
2. Outside boundary of the island are water cells.

For the first condition, whenever we go outside the boundary of the matrix during DFS or BFS, it means that one of the cells of the island is touching an edge; so, the island is not closed. Following code will cover this condition:

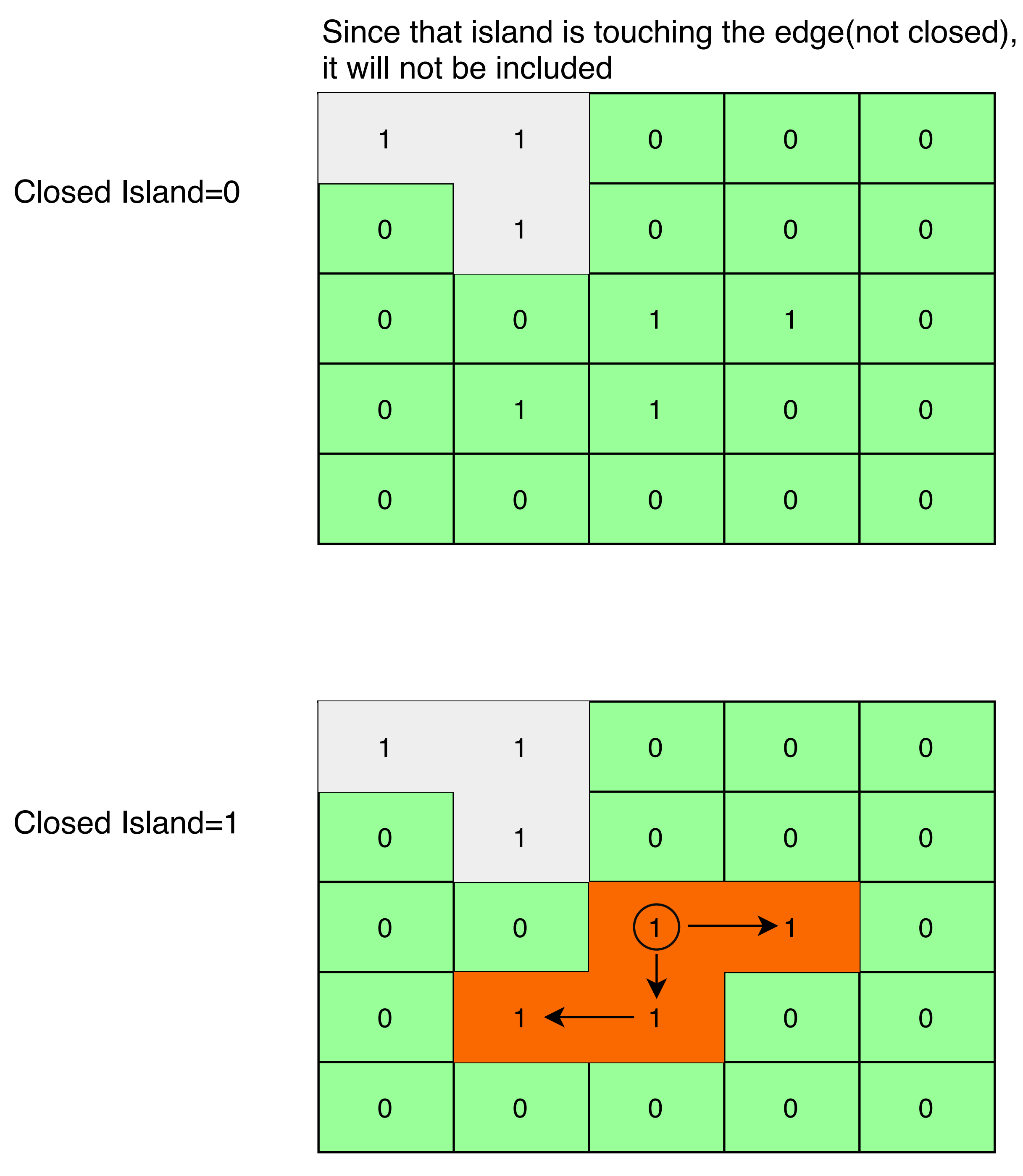
```
if (x < 0 || x >= matrix.length || y < 0 || y >= matrix[0].length)
     return false; // returning false since the island is touching an
edge
```

For the second condition, we need to ensure that all the boundary cells of the island are water. Following code will take care of that:

```
if (matrix[x][y] == 0 || visited[x][y])
     return true; // returning true as the island is surrounded by water
```

Here is the visual representation of the algorithm:

Example: 1

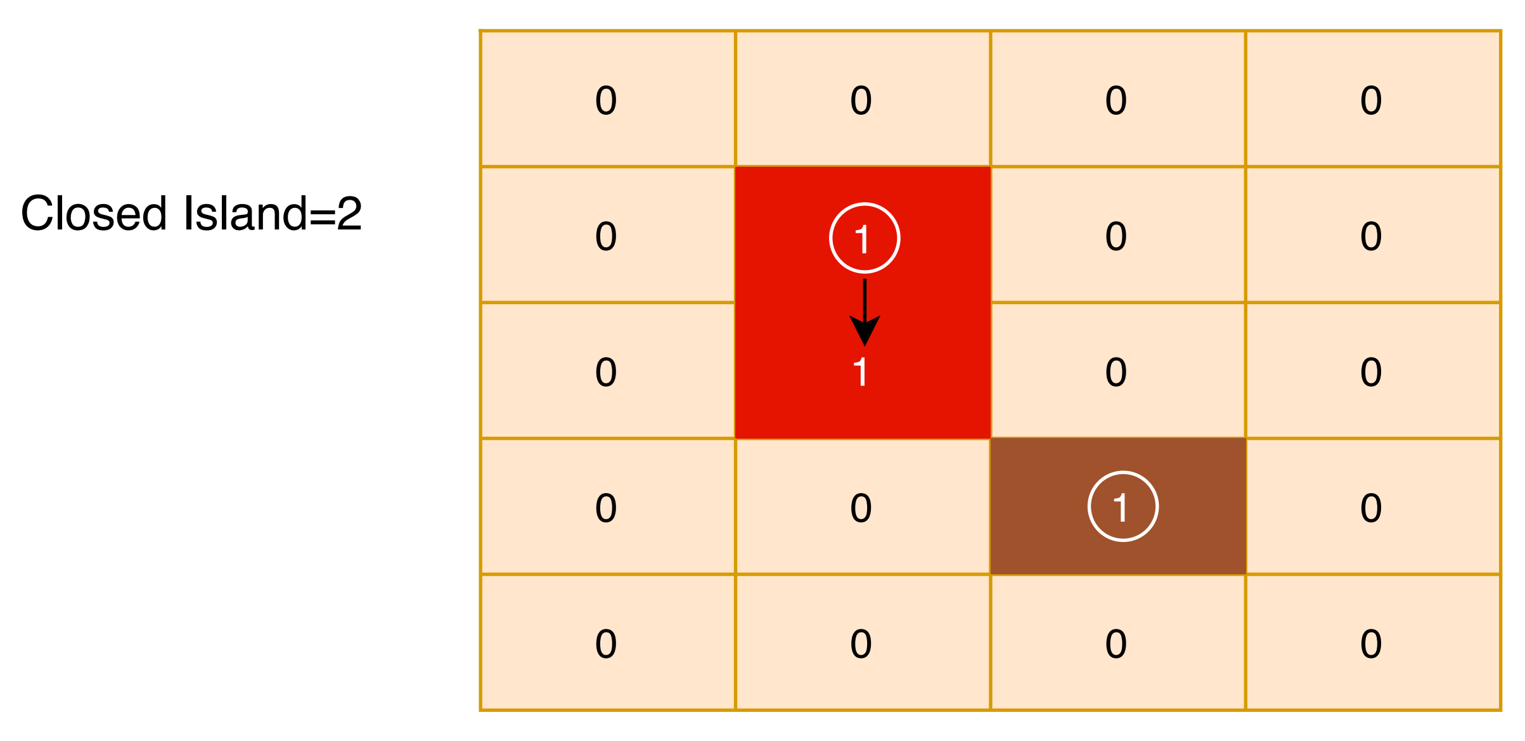


1	1	0	0	0
0	1	0	0	0
0	0	1	1	0
0	1	1	0	0
0	0	0	0	0

1	1	0	0	0
0	1	0	0	0
0	0	①	1	0
0	1	1	0	0
0	0	0	0	0

Output: 1

Example: 2



0	0	0	0
0	①	0	0
0	1	0	0
0	0	①	0
0	0	0	0

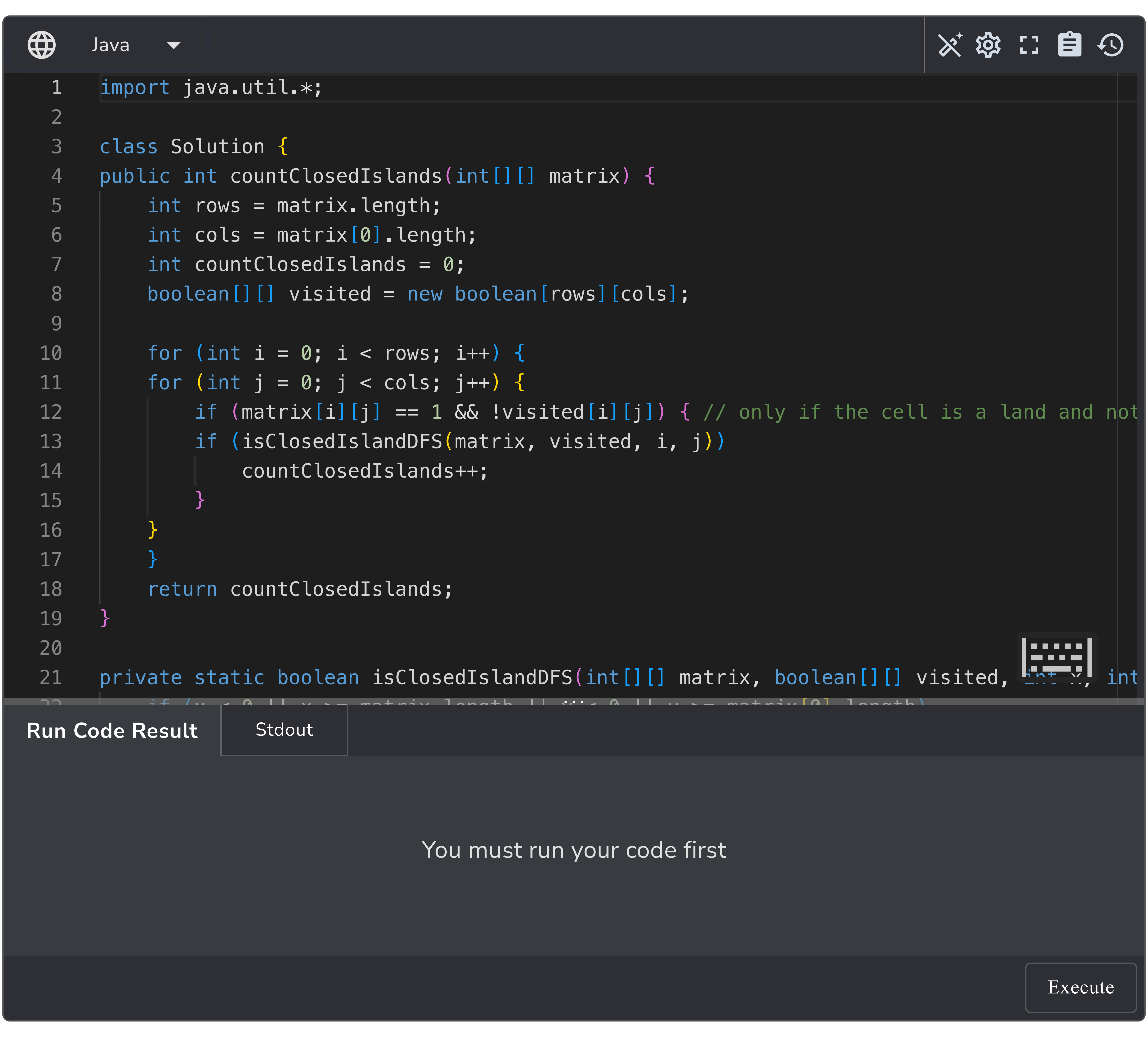
As we can see, no island is touching the edges and hence closed

Output: 2

Number of Closed Islands

Code (DFS)

Here is what our DFS algorithm will look like. We will use a separate array to mark nodes visited.



```
import java.util.*;

class Solution {
public int countClosedIslands(int[][] matrix) {
    int rows = matrix.length;
    int cols = matrix[0].length;
    int countClosedIslands = 0;
    boolean[][] visited = new boolean[rows][cols];

    for (int i = 0; i < rows; i++) {
    for (int j = 0; j < cols; j++) {
        if (matrix[i][j] == 1 && !visited[i][j]) { // only if the cell is a land and not
        if (isClosedIslandDFS(matrix, visited, i, j))
            countClosedIslands++;
        }
    }
    }
    return countClosedIslands;
}

private static boolean isClosedIslandDFS(int[][] matrix, boolean[][] visited, int x, int
```

You must run your code first

Time Complexity

Time complexity of the above algorithm will be $O(M * N)$, where ‘M’ is the number of rows and 'N' is the number of columns of the input matrix. This is due to the fact that we have to traverse the whole matrix to find islands.

Space Complexity

DFS recursion stack can go M*N deep when the whole matrix is filled with '1's. Hence, the space complexity will be $O(M * N)$, where ‘M’ is the number of rows and 'N' is the number of columns of the input matrix.

← Previous
Number Of Closed Islands (Easy)

Next →
Problem Challenge 1 (Easy)

☐ Mark as Completed